

The code serves to manage a linked list structure called `LinkedList`, which stores instances of a bid represented by the `Bid` structure. Each bid consists of a unique identifier (`bidId`), a title, a fund, and an amount. Various methods are provided to manipulate and query the linked list, such as `Append`, `Prepend`, `Remove`, `PrintList`, and `Search`.

The `LinkedList` class is built upon a `Node` structure, each containing a `Bid` object and a reference to the next `Node` in the list. Also, the `LinkedList` class maintains pointers to the head and tail of the list, along with a count of elements in the list.

The `Append` method adds a new `Bid` to the end of the list, while the `Prepend` method inserts a new `Bid` at the beginning. The `Remove` method deletes a `Bid` identified by a specified `bidId`, and the `Search` method locates and returns a `Bid` with the given `bidId`. The `PrintList` method outputs all bids stored in the list.

This code facilitates the management of a dynamic list of bids, offering functionalities for display, search, removal, and addition of bids at either end of the list. Its implementation appears well-structured and facilitates effective management of bid-related tasks.

Pseudocode:

LinkedList class:

1. Initialize head, tail, and size variables.

LinkedList constructor:

1. Initialize head and tail to `nullptr`.

2. Initialize size to 0.

LinkedList destructor:

1. Loop over each node:
 - 1.1. Delete the current node.

Append method:

1. Create a new node with the provided bid.
2. If the list is empty:
 - 2.1. Set head and tail to the new node.
3. Else:
 - 3.1. Set the current tail's next pointer to the new node.
 - 3.2. Update tail to the new node.
4. Increment size.

Prepend method:

1. Create a new node with the provided bid.
2. If the list is not empty:
 - 2.1. Set the new node's next pointer to the current head.
3. Update head to the new node.
4. Increment size.

PrintList method:

1. Start at the head.
2. Loop over each node:
 - 2.1. Print the bid information.
 - 2.2. Move to the next node.

Remove method:

1. If the list is empty, return.
2. If the node to remove is at the head:
 - 2.1. Move head to the next node.
 - 2.2. Delete the current head.
 - 2.3. Decrement size.
 - 2.4. Return.
3. Start at the head.
4. Loop over each node until the end:
 - 4.1. If the next node's bidId matches the provided bidId:
 - 4.1.1. Hold the next node temporarily.
 - 4.1.2. Update the current node's next pointer to skip the next node.
 - 4.1.3. Delete the held node.
 - 4.1.4. Decrement size.
 - 4.1.5. Return.
 - 4.2. Move to the next node.

Search method:

1. If the list is empty, return an empty bid.
2. If the node to search is at the head:
 - 2.1. If the head's bidId matches the provided bidId, return the head's bid.
3. Start at the head.
4. Loop over each node until the end:
 - 4.1. If the current node's bidId matches the provided bidId, return the bid.
 - 4.2. Move to the next node.
5. If no match is found, return an empty bid.